Major Project Report on

AEB-ZKP: An Authentication–Encryption Binding Zero-Knowledge Proof Protocol for Wireless Body Area Networks

Submitted in partial fulfillment of the requirements for the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

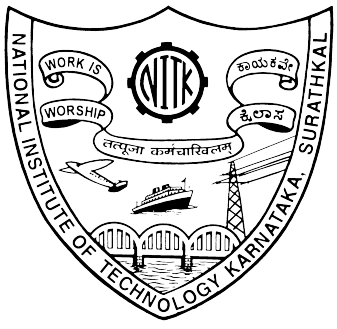
Rithvik Munnur Ujwal (221CS145)

Tanay Praveen Shekokar (221CS159)

Viren Haresh Kundnani (221CS165)

under the guidance of

Dr. Shridhar Sanshi



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY KARNATAKA

SURATHKAL, MANGALORE – 575025

May 11, 2026

DECLARATION

We hereby declare that the Major Project Work Report entitled **"AEB-ZKP: An Authentication–Encryption Binding Zero-Knowledge Proof Protocol for Wireless Body Area Networks"**, submitted to the **National Institute of Technology Karnataka, Surathkal**, for the award of the Degree of **Bachelor of Technology** in **Computer Science and Engineering**, is a *bonafide record of the work carried out by us*. The contents of this report have not been submitted to any other University or Institution for the award of any degree or diploma.

1. **2210536CS145, Rithvik Munnur Ujwal**
2. **2210031CS159, Tanay Praveen Shekokar**
3. **2210058CS165, Viren Haresh Kundnani**

Department of Computer Science and Engineering

Place: NITK, Surathkal
Date: May 11, 2026

CERTIFICATE

This is to *certify* that the Major Project Report entitled ***"AEB-ZKP: An Authentication–Encryption Binding Zero-Knowledge Proof Protocol for Wireless Body Area Networks"*** submitted by:

Sl. No.	Register Number	Name of Student
1.	2210536CS145	Rithvik Munnur Ujwal
2.	2210031CS159	Tanay Praveen Shekokar
3.	2210058CS165	Viren Haresh Kundnani

as the record of the work carried out by them, is *accepted as the B.Tech. Major Project report submission* in fulfillment of the requirement for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering**.

Guide:
Dr. Shridhar Sanshi
Department of CSE, NITK Surathkal

Chairman – DUGC
Dr. B. R. Chandavarkar
Department of CSE, NITK Surathkal

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our guide, **Dr. Shridhar Sanshi**, for his invaluable guidance, expertise, and encouragement throughout the course of this project. His guidance, consistent support, and patient mentoring played an important role in shaping this work and improving our understanding of the subject. We sincerely thank him for the time, effort, and dedication he invested in supervising our project.

We are also grateful to the Head of the Department, **Dr. B. R. Chandavarkar**, for his support and encouragement throughout our academic journey. His leadership and commitment to maintaining a strong academic environment have greatly contributed to fostering learning, research, and innovation within the department.

We would also like to thank the faculty members of the **Department of Computer Science and Engineering, NITK Surathkal**, for their guidance, valuable feedback, and engaging discussions throughout our course of study. Their support and insights have helped broaden our understanding and contributed to the development of this project.

We would also like to thank our classmates and friends for their support, encouragement, and helpful discussions throughout this journey. Their motivation and shared experiences made the process more enjoyable and helped us stay motivated during the course of this project.

ABBREVIATIONS

Abbreviation	Full Form
AEB-ZKP	Authentication–Encryption Binding Zero-Knowledge Proof
AES	Advanced Encryption Standard
AES-GCM	AES in Galois/Counter Mode
CDH	Computational Diffie-Hellman
DH	Diffie-Hellman
DoS	Denial of Service
ECG	Electrocardiogram
ECDH	Elliptic Curve Diffie-Hellman
EVP	Envelope (OpenSSL high-level API)
HKDF	HMAC-based Key Derivation Function
ID	Identity
IEEE	Institute of Electrical and Electronics Engineers
IV	Initialization Vector
$K_{0,N}$	Symmetric key pre-shared between the node and the server
MAC	Message Authentication Code
MitM	Man-in-the-Middle
NS-3	Network Simulator 3
PKC	Public Key Cryptography
RPi	Raspberry Pi
SHA	Secure Hash Algorithm
SK	Session Key
SPDL	Security Protocol Description Language (Scyther)
SpO_2	Blood Oxygen Saturation
TLV	Type-Length-Value
UDP	User Datagram Protocol
WBAN	Wireless Body Area Network
WSN	Wireless Sensor Network
ZKP	Zero-Knowledge Proof

ABSTRACT

Wireless Body Area Networks (WBANs) are becoming an important part of modern healthcare systems by enabling the continuous monitoring and transmission of sensitive physiological data through wearable or implantable sensor nodes. However, the constrained nature of WBAN devices — including limited battery capacity, restricted memory, and low-power processing capabilities — makes the use of conventional public-key cryptographic techniques impractical. Although several lightweight authentication schemes have been proposed for WBANs, many of them either lack rigorous formal security validation or are unable to provide both forward secrecy and strong agreement properties simultaneously.

This project presents the design, formal verification, and simulation of the Authentication–Encryption Binding Zero-Knowledge Proof (AEB-ZKP) protocol, a lightweight approach for mutual authentication and session key establishment in WBAN environments. The protocol completes the authentication process in just three message exchanges between a sensor node and a server. It combines a pre-shared symmetric key for initial handshake authentication with a Diffie-Hellman key exchange to derive a fresh session key for every communication session.

To achieve zero-knowledge proof functionality, the sensor node proves possession of the shared Diffie-Hellman secret without revealing the secret itself during communication. The protocol also incorporates a session-binding mechanism to prevent multi-session confusion attacks. In addition, only a hash of the shared secret is transmitted, helping preserve forward secrecy even if long-term credentials are compromised later.

The protocol was formally verified using the Scyther model checker under the Dolev-Yao attacker model, with all seven claimed security properties — data confidentiality, session key secrecy, ephemeral secret secrecy, aliveness, weak agreement, non-injective agreement, and synchronisation — verified for both the node and server roles. The protocol was further evaluated through simulation in the NS-3 network simulator using production-grade OpenSSL cryptographic primitives, including Curve25519 ECDH, AES-128-GCM, and SHA-256, over an IEEE 802.15.4-compatible communication channel. In addition, a hardware prototype was implemented using Raspberry Pi devices and Python to assess the protocol under practical computational conditions. The evaluation results indicate that the protocol introduces an authentication-phase communication overhead of approximately 1808 bits, while keeping the total computation time at the sensor node below 500 μs.

Keywords— Authentication, Curve25519, Diffie-Hellman, Forward Secrecy, NS-3, Raspberry Pi, Scyther, Wireless Body Area Networks, Zero-Knowledge Proof.

CONTENTS

LIST OF FIGURES i
LIST OF TABLES ii

1. INTRODUCTION 1
1.1 Overview 1
1.2 Motivation 1
1.3 Objectives of the Project 2

2. LITERATURE REVIEW 3
2.1 Background and Related Works 3
2.2 Outcome of Literature Review 6
3. PROPOSED METHODOLOGY 7
3.1 System Model 7
3.2 Protocol Message Sequence 9
3.3 Step-by-Step Protocol Description 9
3.4 Formal Verification with Scyther 11
3.5 NS-3 Network Simulation 12
3.6 Hardware Prototype on Raspberry Pi 14

4. RESULTS AND ANALYSIS 18
4.1 Scyther Verification Results 18
4.2 NS-3 Simulation Results 19
4.3 Hardware Prototype Results 21
4.4 Comparison with Related Work 23

5. CONCLUSIONS AND FUTURE WORK 26
5.1 Conclusions 26
5.2 Future Work 26

REFERENCES 27

LIST OF FIGURES

3.1 System Architecture 7

3.2 NS-3 Simulation Topology 13

3.3 Raspberry Pi Hardware Prototype Setup 16

4.1 Scyther Output — All Security Claims Verified for AEB-ZKP 19

4.2 NS-3 Simulation Console Output and Statistics 20

4.3 Hardware Simulation - Node Side Output 22

4.4 Hardware Simulation - Server Side Output 23

LIST OF TABLES

2.1 Summary of Related WBAN Authentication Schemes 5

3.1 AEB-ZKP Protocol Message Sequence 9

3.2 Scyther Security Claims 12

3.3 NS-3 Simulation Parameters 13

4.1 Communication Overhead per Protocol Step 19

4.2 Comparison with Related WBAN Authentication Protocols 24

CHAPTER 1

INTRODUCTION

1.1 Overview

Wireless Body Area Networks (WBANs) are short-range wireless systems made up of small sensor nodes that are either worn on or implanted inside the human body. These sensors continuously monitor physiological parameters such as heart rate, blood oxygen saturation (SpO_2), body temperature, electrocardiogram (ECG) signals, and motion data from accelerometers. The collected information is then transmitted to a central server or medical gateway for real-time analysis, monitoring, and clinical decision-making.

Unlike conventional wireless sensor networks, WBANs operate in a highly sensitive and safety-critical environment. The data transmitted through these networks contains personal medical information, making confidentiality and data integrity essential requirements. Any compromise in security can directly affect patient privacy and, in some cases, patient safety. At the same time, WBAN devices are highly resource-constrained, with limited battery capacity, very small memory availability, and low-power processors operating at only a few megahertz. Because of these hardware limitations, deploying traditional public-key cryptographic infrastructures in WBANs is often impractical.

This project focuses on addressing the challenge of designing a security protocol that remains lightweight enough for constrained WBAN devices while still providing strong and formally verifiable security guarantees. The proposed approach aims to achieve essential properties such as mutual authentication, session key secrecy, and forward secrecy, while keeping the protocol communication overhead and the number of message exchanges minimal.

1.2 Motivation

The increasing use of WBANs in hospitals and home-healthcare environments has made security an important challenge. Sensitive physiological data transmitted over wireless communication channels is vulnerable to several types of attacks, including eavesdropping, replay attacks, man-in-the-middle impersonation, and malicious data injection. A compromised WBAN session could allow an attacker to alter medical readings, inject false data into clinical records, or decrypt previously recorded sessions to reveal a patient's medical history.

Existing authentication schemes for WBANs fall into two broad categories. Symmetric-key schemes

are energy efficient but typically lack forward secrecy and formal agreement properties. Public-key and elliptic-curve schemes provide stronger security guarantees but impose prohibitive computational and energy costs on constrained sensors. Zero-Knowledge Proof (ZKP) based schemes are a promising middle ground — a node can prove knowledge of a secret without revealing it — but existing ZKP-based WBAN protocols such as TinyZKP [4] and BANZKP [3] rely on large pre-distributed key sets and provide no formal verification of their security properties.

The AEB-ZKP protocol designed in this project aims to close these gaps: a three-message authenticated key exchange that provides machine-verified security properties, implemented with production-grade cryptographic primitives, and validated through both network simulation and hardware prototyping.

1.3 Objectives of the Project

1. Design the AEB-ZKP protocol providing mutual authentication, session key establishment, forward secrecy, and session binding within three protocol messages.
2. Formally verify all security properties using the Scyther model checker.
3. Implement and evaluate the protocol in the NS-3 network simulator using production-grade OpenSSL cryptographic primitives (Curve25519, AES-128-GCM, SHA-256).
4. Develop a hardware prototype using a Raspberry Pi device as a node to validate the protocol under real computation costs.
5. Compare the protocol's security properties and communication overhead against other WBAN authentication schemes.

CHAPTER 2

LITERATURE REVIEW

2.1 Background and Related Works

The design of lightweight authentication protocols for Wireless Body Area Networks has been an active research area for over a decade, with significant recent activity driven by the rise of Medical 4.0 and the Internet of Health Things (IoHT). Early works such as TinySec [1], MiniSec [2], and TinyZKP [4] established the foundational requirements and constraints, while more recent work has progressively strengthened the security properties achievable within constrained WBAN hardware. This section surveys the most relevant recent contributions, contextualising the limitations that the AEB-ZKP protocol addresses.

The **BANZKP** protocol [3] is the most closely related prior work. It combines a ZKP-based challenge-response mechanism with a commitment scheme to achieve mutual authentication in WBANs, evaluated in the OMNeT++ simulator. BANZKP reports 17× and 5× speedups over TinyZKP and W-ECDSA respectively, with 94.11% and 80% lower energy consumption. However, BANZKP provides no formal verification of its security claims and its security analysis is limited to informal argument against named attacks.

TinyZKP [4] is the first ZKP-based scheme specifically designed for WBANs, allowing a receiver to verify that data originates from a specific sender without leaking secret information. However, it requires 20 private and 20 public keys per node, imposing significant memory overhead, and uses ECDSA signatures of at least 320 bits for message signing.

Zhong et al. (2023) [5] proposed an anonymous authentication scheme for WBANs based on the Chinese Remainder Theorem, published in the journal Mathematics. The scheme achieves anonymity and untraceability by using dynamic pseudonyms derived from CRT operations, avoiding the transmission of real identities in any message. The authors verify security using BAN logic. While the scheme is creative in its use of number-theoretic primitives for anonymity, CRT-based operations are more computationally expensive than simple hash-and-XOR schemes on 8-bit microcontrollers, and the paper does not evaluate communication overhead in bits — making direct comparison with related protocols difficult.

Wang et al. (2023) [6] proposed a lightweight zero-knowledge authentication scheme for IoT embedded devices, published in Computer Networks (Elsevier). The scheme uses ZKP to allow a device to prove possession of a secret to a verifier without revealing it, combining ZKP with a MAC-based

challenge-response mechanism. The work is relevant because it addresses the same ZKP-in-IoT problem space as AEB-ZKP, and demonstrates that ZKP-based authentication on embedded hardware is practically feasible. However, the scheme is designed for general IoT rather than WBAN specifically, and does not consider the multi-session agreement properties (Niagree, Nisynch) that are critical in concurrent WBAN deployments.

Wu et al. (2025) [9] presented an improved and untraceable lightweight authentication and key agreement scheme for WBANs, published in Concurrency and Computation: Practice and Experience. The work identifies that Narwal et al.'s SAMAKA protocol is vulnerable to sensor node capture attacks, lacks session unlinkability, and does not provide forward secrecy. The proposed improvement uses only hash functions and XOR operations, and the authors formally verify the scheme using BAN logic and the AVISPA tool. Performance analysis shows significant advantages in computational and communication cost over comparable schemes. This is one of the most recent directly comparable papers. A key limitation of BAN logic and AVISPA is that they provide weaker formal guarantees compared to Scyther. BAN logic cannot model multi-session attacks, while AVISPA supports verification only for bounded sessions. Consequently, neither can exhaustively verify agreement properties such as Nisynch under unbounded concurrent executions.

Subramani et al. (2025) [10] proposed a lightweight and physically secure authentication scheme for WBANs integrated with Fog Computing, published in Scientific Reports (Nature). The protocol uses PUFs and a fog-assisted architecture to shift authentication tasks away from resource-constrained sensor nodes, achieving reductions of 64.33% in computational overhead and 29.58% in communication overhead compared to earlier schemes. The design follows a two-phase model consisting of registration and anonymous authentication, with the aim of improving privacy and the resilience of the system. Despite these improvements, the approach depends on the availability of nearby fog infrastructure, which may not be practical in remote or emergency healthcare settings. In addition, the protocol relies on the stability assumptions of PUFs and does not include symbolic verification under the Dolev-Yao attacker model.

Khandait et al. (2024) [8] proposed a lightweight Zero-Knowledge Proof-based authentication framework for IoT devices, presented at an international conference in 2024. The scheme combines ZKP with edge computing to enable low-latency authentication for resource-constrained devices, demonstrating the practical feasibility of ZKP-based approaches in IoT environments. While the work supports the motivation behind the AEB-ZKP protocol, it does not address WBAN-specific challenges such as session agreement properties or provide detailed bit-level overhead analysis.

Belfaik et al. (2024) [7] conducted a comparative study of protocol security verification tools — AVISPA, Scyther, ProVerif, and Tamarin — published in the proceedings of ICDTA 2024. This survey is directly relevant to our verification methodology choice. The authors conclude that Scyther provides the strongest guarantees for symbolic multi-session analysis of authentication and agreement properties, while Tamarin extends this to protocols with algebraic operators (such as Diffie-Hellman). ProVerif provides bounded verification in the applied pi-calculus. This comparative analysis validates the choice of Scyther as the primary verification tool for AEB-ZKP's authentication and agreement claims.

Table 2.1: Summary of Recent WBAN Authentication Literature (2022–2025)

Author & Year	**Approach**	**Key Strength**	**Limitation**
TinyZKP [4]	ZKP challenge-response	No secret revealed in exchange	20 keys per node; ECDSA signing overhead; no formal verification
BANZKP [3]	ZKP + commitment scheme	Low energy; fast execution	No formal verification; informal security argument only
Zhong et al., 2023 [5]	Chinese Remainder Theorem for dynamic pseudonyms; BAN logic	Strong anonymity and untraceability; no real ID on wire	CRT costlier than hash/XOR on MCUs; no bit-level overhead analysis
Wang et al., 2023 [6]	ZKP + MAC challenge-response for embedded IoT devices	Practical ZKP on embedded hardware demonstrated	General IoT, not WBAN-specific; no multi-session agreement analysis
Khandait et al., 2024 [8]	ZKP + edge computing for low-latency IoT authentication	Practical ZKP feasibility on constrained devices shown	No WBAN-specific overhead or session-agreement analysis
Wu et al., 2025 [9]	Hash + XOR; fixes SAMAKA; BAN logic + AVISPA verification	Most recent directly comparable work; lightweight and untraceable	BAN logic / AVISPA cannot verify multi-session Nisynch; no forward secrecy via DH
Subramani et al., 2025 [10]	PUF + Fog Computing; two-phase WBAN authentication	64.33% less computation; 29.58% less communication than prior work	Requires fog infrastructure; PUF stability assumption; no symbolic verification
AEB-ZKP (Ours)	DH (Curve25519) + ZKP hash proof + AES-128 + SHA-256	Scyther-verified Niagree, Nisynch, forward secrecy; no PUF or fog required	Slightly higher overhead than hash/XOR-only schemes

2.2 Outcome of Literature Review

The survey of recent literature reveals that while the field has made substantial progress in lightweight WBAN authentication, three critical gaps persist across essentially all recent proposals:

1. **Reliance on informal or bounded formal verification.** The dominant verification tools in recent WBAN authentication papers are BAN logic and AVISPA. BAN logic is a modal logic that reasons about beliefs rather than trace properties, and cannot capture multi-session attack scenarios. AVISPA operates on a bounded number of protocol sessions, meaning it cannot exhaustively verify properties such as non-injective agreement (Niagree) or synchronisation (Nisynch) across arbitrarily many concurrent sessions. Scyther, which is used in this work for verifying AEB-ZKP, supports unbounded symbolic verification and is among the few widely used tools capable of conclusively verifying or falsifying Nisynch, a property essential for preventing session confusion attacks. The comparative study by Belfaik et al. [7] further highlights the stronger verification capabilities offered by Scyther compared to other commonly used formal verification tools.

2. **Forward secrecy achieved without Diffie-Hellman.** Recent hash-and-XOR schemes (Xie et al.[11], Wu et al.) claim forward secrecy but achieve it through key update mechanisms or hash chains rather than fresh ephemeral key exchange. These approaches require either a trusted registration centre, stored state between sessions, or sequential session identifiers — all of which introduce additional assumptions. The AEB-ZKP approach uses a fully fresh Curve25519 DH exchange per session, achieving forward secrecy without any stored state or session counter.

3. **Physical security assumptions (PUF reliance).** Several recent schemes (Subramani et al. [10]) rely on PUFs to authenticate node identity without a pre-shared key. While PUFs are conceptually elegant, they require manufacturing support, are sensitive to environmental conditions in body-worn devices, and are subject to machine-learning modelling attacks. AEB-ZKP uses a simple pre-shared symmetric key provisioned at registration, which is more robust for implantable and wearable WBAN nodes where hardware-level physical assumptions are difficult to guarantee.

These observations motivate the AEB-ZKP protocol design: a Diffie-Hellman based key exchange providing genuinely fresh session keys, a ZKP-style proof of knowledge without revealing the shared secret, and Scyther-verified Niagree and Nisynch properties — a combination not achieved by any of the surveyed recent works.

CHAPTER 3

PROPOSED METHODOLOGY

3.1 System Model

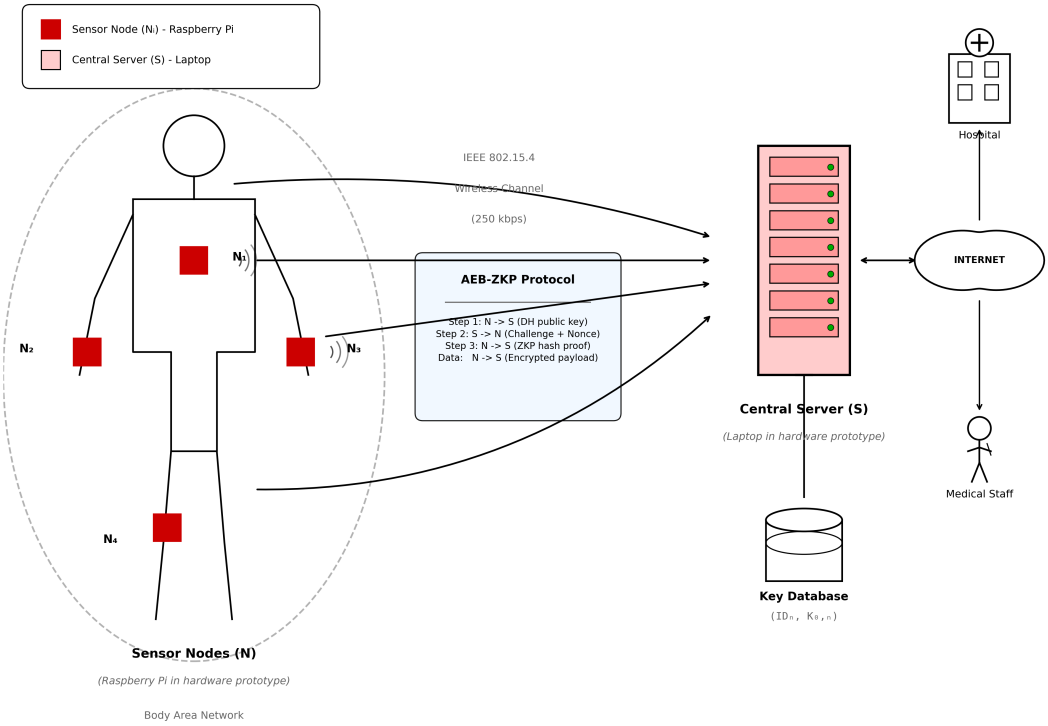


Fig. 3.1: Architecture diagram of the proposed model

Network Architecture

The AEB-ZKP protocol operates in a star topology typical of Wireless Body Area Networks. The architecture consists of multiple resource-constrained sensor nodes communicating with a single central server (medical gateway or base station). Each sensor node collects physiological data and transmits it securely to the server for processing, storage, and clinical decision support.

The system includes the following components:

- **Sensor Nodes** $(N_1, N_2, \ldots, N_n)$ — Lightweight WBAN devices implanted on the patient's body. Each node is identified by a unique 2-byte numeric identity ID_{N_i} and is responsible for collecting

specific physiological signals (e.g., ECG, SpO_2, temperature, accelerometer data).

- **Central Server** S — A trusted medical gateway or base station with identity ID_S. The server acts as the central coordinator for all authentication sessions and maintains the master key database for all registered nodes.

Initial Configuration and Key Management

Before deployment, the system undergoes a secure registration phase conducted out-of-band (e.g., during device provisioning at a healthcare facility):

1. **Node Registration:** Each sensor node N_i is assigned a unique 2-byte numeric identifier ID_{N_i} and a 16-byte pre-shared symmetric key K_{0,N_i}.
2. **Key Storage at Server:** The central server maintains a secure key database that stores the tuple (ID_{N_i}, K_{0,N_i}) for every registered node. This allows the server to look up the appropriate pre-shared key when a node initiates an authentication session.
3. **Key Storage at Node:** Each sensor node stores only its own pre-shared key K_{0,N_i} and the server's identity ID_S. Nodes do not store keys belonging to other nodes, minimising the impact of node compromise.
4. **Server Identity:** The server's identity ID_S is a 2-byte numeric value known to all nodes, enabling nodes to verify they are communicating with the legitimate server.

Protocol Participants

During each authentication session, the protocol involves two principals:

- **Node** N — a WBAN sensor with identity ID_N and pre-shared key $K_{0,N}$. The node generates a fresh ephemeral Diffie-Hellman keypair (p_N, V^{p_N}) at the start of each session, where p_N is the private exponent and V^{p_N} is the corresponding 32-byte Curve25519 public key.
- **Server** S — the central gateway with identity ID_S. It retrieves $K_{0,N}$ from its key database upon receiving an authentication request, generates its own ephemeral DH keypair (q_S, V^{q_S}) per session, and produces a fresh 8-byte randomness nonce R to ensure session uniqueness.

Key Separation Principle

The protocol enforces strict separation between authentication keys and session keys:

- The pre-shared key $K_{0,N}$ is used *exclusively* for encrypting handshake messages (Steps 1–3). It is never used to encrypt application data.

- All sensor data is protected under a fresh session key SK derived during each protocol run using the Diffie-Hellman shared secret:

$$SK = H(V^{p_N \cdot q_S} \parallel ID_N \parallel ID_S \parallel R)$$

- This separation ensures that compromise of $K_{0,N}$ does not expose previously transmitted session data (forward secrecy), as the attacker cannot recover the ephemeral DH exponents needed to reconstruct past session keys.

3.2 Protocol Message Sequence

Table 3.1: AEB-ZKP Protocol Message Sequence

Step	**Direction**	**Message**	**Purpose**
Step 1	$N \rightarrow S$	$E(K_{0,N},\ [ID_N \parallel V^{p_N}])$	Node commits its DH public half
Step 2	$S \rightarrow N$	$E(K_{0,N},\ [ID_S \parallel V^{q_S} \parallel R \parallel V^{p_N}])$	Server responds; echoes V^{p_N} for session binding
Step 3	$N \rightarrow S$	$E(K_{0,N},\ [ID_N \parallel H(V^{p_N \cdot q_S})])$	Node proves knowledge of shared secret
Step 4	Server (internal)	Verify H(shared); derive SK	Server checks proof and computes session key
Data	$N \rightarrow S$	$E(SK,\ [ID_N \parallel \text{DATA}])$	Payload encrypted under derived session key

The session key is derived as:

$$SK = H(V^{p_N \cdot q_S} \parallel ID_N \parallel ID_S \parallel R)$$

3.3 Step-by-Step Protocol Description

Step 1 — Authentication Initiation

Node N generates a fresh ephemeral secret p_N and computes its Diffie-Hellman public half $V^{p_N} = g^{p_N} \bmod p$. It encrypts ID_N concatenated with V^{p_N} under the pre-shared key $K_{0,N}$ using AES-128-GCM with a fresh random IV, and transmits the ciphertext to S:

$$N \rightarrow S : E(K_{0,N},\ [ID_N \parallel V^{p_N}])$$

The server decrypts using $K_{0,N}$, verifies that ID_N corresponds to a registered node, and stores V^{p_N} for use in Step 4.

Step 2 — Server Challenge

The server generates its own ephemeral secret q_S, computes $V^{q_S} = g^{q_S} \bmod p$, and generates a random 8-byte freshness nonce R. It responds with ID_S, V^{q_S}, R, and the echoed V^{p_N}, all encrypted under $K_{0,N}$:

$$S \to N : E(K_{0,N},\ [ID_S \parallel V^{q_S} \parallel R \parallel V^{p_N}])$$

Upon receipt, N verifies that the echoed V^{p_N} matches the value it sent in Step 1. This session binding check prevents multi-session confusion attacks: a response intended for a different protocol run will carry a mismatched V^{p_N} and will be rejected.

Step 3 — Proof of Knowledge

N computes the shared DH secret:

$$\text{shared} = V^{p_N \cdot q_S} = (V^{q_S})^{p_N}$$

Instead of transmitting the raw shared secret, N sends only its SHA-256 hash, serving as a zero-knowledge proof that N has correctly computed the shared value without revealing it:

$$N \to S : E(K_{0,N},\ [ID_N \parallel H(V^{p_N \cdot q_S})])$$

Forward secrecy is achieved by transmitting $H(\text{shared})$ instead of the shared secret directly. If $K_{0,N}$ is compromised after the session and an attacker decrypts this message, they obtain only the hash value. Since SHA-256 is a one-way function, they cannot invert it to recover shared and therefore cannot reconstruct SK.

Simultaneously with Step 3, the node sends the data payload under the session key (optimistic sending). This avoids an additional round trip, which is critical for latency-sensitive WBAN applications.

Step 4 — Verification and Key Derivation

This is an internal computation at the server; no message is transmitted. S independently computes:

$$\text{shared} = V^{p_N \cdot q_S} = (V^{p_N})^{q_S}$$

and verifies that $H(\text{shared})$ matches the value received in Step 3. If the check passes, both parties derive the session key identically using the same SHA-256 inputs:

$$SK = H(\text{shared} \parallel ID_N \parallel ID_S \parallel R)$$

If verification fails, the server silently discards the data and aborts.

3.4 Formal Verification with Scyther

Scyther is a model checker for security protocols that operates under the Dolev-Yao attacker model. The attacker is assumed to control the network: it can intercept, replay, modify, and inject messages, but cannot break cryptographic primitives. Scyther performs an exhaustive symbolic search over all possible attack traces and either proves that a claimed property holds for all executions, or produces a concrete counterexample trace.

Modelling Diffie-Hellman in Scyther

Scyther does not natively support modular exponentiation. The DH structure was encoded using nested hash terms that preserve the one-way commitment property of exponentiation:

- V^{p_N} is modelled as $\texttt{Hash}(p_N)$
- $V^{p_N \cdot q_S}$ is modelled as $\texttt{Hash}(\texttt{Hash}(p_N),\ q_S)$
- $H(V^{p_N \cdot q_S})$ (the Step 3 payload) becomes $\texttt{Hash}(\texttt{Hash}(\texttt{Hash}(p_N),\ q_S))$
- SK is modelled as $\texttt{Hash}(\texttt{Hash}(\texttt{Hash}(\texttt{Hash}(p_N), q_S)), N),\ \texttt{Hash}(S, R))$

Scyther's one-way hash semantics prevent the attacker from computing inner terms from outer ones, correctly modelling both the one-way and commitment properties of discrete exponentiation.

An important distinction is that in the Scyther model, the raw ephemeral values p_N and q_S are exchanged inside the $K_{0,N}$-encrypted messages rather than as computed public halves. Both parties therefore converge on the same hash term directly. The Dolev-Yao attacker cannot break the encryption and therefore cannot access these values. This correctly verifies the authentication and agreement properties under the assumption that $K_{0,N}$ is uncompromised. The hardness of the discrete logarithm problem — the property that $g^{p_N \cdot q_S}$ cannot be computed from g^{p_N} and g^{q_S} alone — is a computational property outside Scyther's symbolic scope.

Table 3.2: Scyther Security Claims and Verification Results

Property (Claim)	**Result**	**Meaning**
Secret(DATA)	**OK**	Sensor payload is not learnable by the attacker
Secret(SK)	**OK**	Session key remains unknown to the attacker
Secret(p_N, q_S)	**OK**	Ephemeral DH exponents are not exposed
Alive	**OK**	The other party has recently participated in the protocol
Weakagree	**OK**	Both parties agree on each other's identities
Niagree	**OK**	Both parties agree on all exchanged message values
Nisynch	**OK**	Both parties agree on the full message sequence and ordering

3.5 NS-3 Network Simulation

Following formal verification, the protocol was implemented in NS-3 (version 3.45), a discrete-event network simulator widely used for evaluating networking protocols under realistic channel conditions. The simulation directly integrates OpenSSL via its EVP API, providing genuine cryptographic computation rather than simulated stand-ins.

Simulation Topology and Parameters

Table 3.3: NS-3 Simulation Parameters

Parameter	**Value**
Topology	Single-hop point-to-point (1 sensor node, 1 server)
Link data rate	250 kbps (IEEE 802.15.4 PHY rate)
Propagation delay	5 ms (body-area channel)
Transport	UDP sockets
Node identity	2-byte numeric ID (reduces string overhead by 11–13 bytes per message)
DH primitive	Curve25519 ECDH via `EVP_PKEY_X25519` (32-byte public keys, 128-bit security)
Symmetric cipher	AES-128-GCM via `EVP_aes_128_gcm` (12-byte fresh IV per message)
Hash / KDF	SHA-256 via `EVP_sha256`
Timing instrument	`std::chrono::high_resolution_clock` (μs resolution)

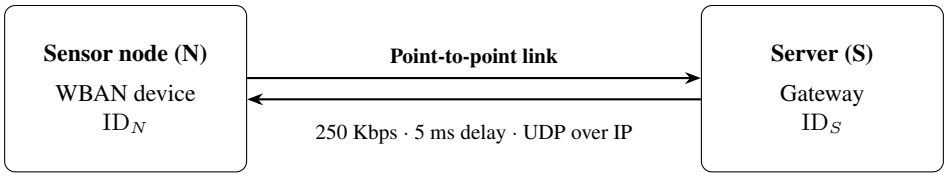


Fig. 3.2: NS-3 simulation topology — single-hop WBAN

Cryptographic Primitives

The simulation uses three production-grade OpenSSL primitives:

- **Curve25519 ECDH** — generates a 32-byte ephemeral public key per session. Both the node and server independently derive the same 32-byte shared secret. Curve25519 provides 128-bit security and replaces the 768-bit Diffie-Hellman used in an earlier prototype, reducing public key size from 96 bytes to 32 bytes.
- **SHA-256** — used in two roles: hashing the shared secret for the Step 3 zero-knowledge proof, and deriving the session key $SK = H(\text{shared} \| ID_N \| ID_S \| R)$.
- **AES-128-GCM** — encrypts all protocol messages. A fresh random IV is generated per message via `RAND_bytes` and prepended to the ciphertext. Steps 1–3 use $K_{0,N}$; the data message uses the first 16 bytes of SK.

Before the simulation starts, a warm-up function is executed to trigger the lazy initialization of the cryptographic algorithms by OpenSSL. This ensures that the recorded timings represent the actual per-operation computation cost rather than including one-time library initialization overhead.

Optimistic Sending

To satisfy the low-latency requirements of WBANs, the node transmits the data payload at the same simulation time as Step 3 instead of waiting for Step 4 verification from the server. If the Step 4 verification later fails, the server silently discards the encrypted data. This design does not compromise security, since the payload is already encrypted using the session key SK, which the server can derive independently only after successful verification.

3.6 Hardware Prototype on Raspberry Pi

To complement the NS-3 simulation and evaluate the protocol under real hardware computation costs, a prototype implementation was developed using a Raspberry Pi as the WBAN sensor node and a laptop as the server, with communication carried out over a local network using UDP sockets.

Hardware Configuration

The prototype uses a heterogeneous setup that closely reflects a realistic WBAN deployment:

- **Node (Raspberry Pi 4 Model B)** — A single-board ARM-based computer running Raspberry Pi OS and executing the `node.py` application. The Raspberry Pi acts as the constrained WBAN sensor node, responsible for initiating authentication sessions and transmitting simulated physiological data.

- **Server (Laptop)** — A standard laptop running the `server.py` application, representing the medical gateway or base station responsible for receiving and processing data from WBAN nodes. The higher computational capability of the laptop reflects the typical asymmetry in real WBAN deployments, where the server possesses significantly greater resources than the constrained sensor nodes.

The two devices communicate over a standard Wi-Fi network connection. Both devices share a 16-byte pre-shared key ($K_{0,N}$) stored in a common key file (`k0n.key`) and distributed securely through an out-of-band provisioning process.

Software Implementation

All cryptographic operations are implemented using the Python `cryptography` library, which is backed by OpenSSL, matching the cryptographic primitives used in the NS-3 simulation:

- `X25519PrivateKey` — Curve25519 keypair generation and ECDH shared secret derivation.
- `HKDF` with SHA-256 — session key derivation from the shared secret, node ID, server ID, and freshness nonce R.
- `AESGCM` — authenticated encryption of all protocol messages (Steps 1–3) and the data payload using AES-128-GCM.

The node application (`node.py`) generates simulated WBAN sensor data including heart rate, blood oxygen saturation (SpO_2), body temperature, and accelerometer readings. Each session runs the complete four-step protocol: the node sends Step 1, receives Step 2 from the server, verifies the session binding by checking the echoed V^{p_N}, and then sends Step 3 and the data message using optimistic sending.

The server application (`server.py`) listens on a configurable UDP port, processes incoming protocol messages, verifies the hash-protected proof in Step 3, derives the session key, and decrypts the received sensor data.

Test Procedure

The hardware prototype was tested as follows:

1. The pre-shared key was generated using `python node.py --generate-key` and copied to both the Raspberry Pi and the laptop.
2. The server was started on the laptop: `python server.py --port 9999 --verbose`
3. The node was started on the Raspberry Pi, pointing to the laptop's IP address: `python node.py --server-ip <laptop_ip> --port 9999 --num-sessions 100 --verbose`

4. Per-operation timing measurements were collected using `time.perf_counter()` at each protocol step, with microsecond resolution.

5. Multiple sessions were executed with different simulated sensor data types, including ECG, vitals, and accelerometer data, to verify correct operation across varying payloads.

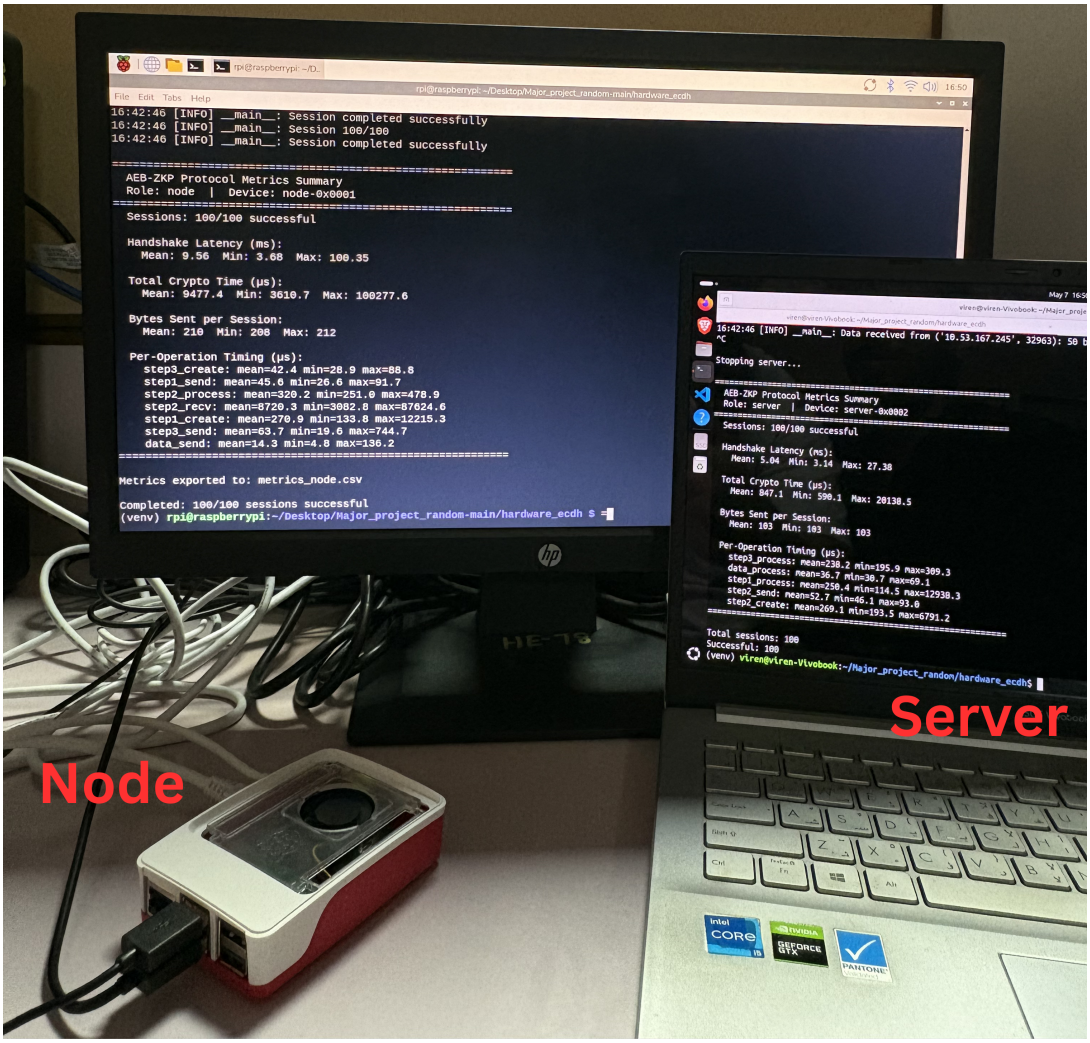


Fig. 3.3: Hardware simulation setup using a Raspberry Pi [Node] and Laptop [Server]

Why This Setup Complements NS-3

The NS-3 simulation is used to validate the protocol at the network level, including message flow, packet structure, and communication overhead under controlled channel conditions. In contrast, the Raspberry Pi to laptop experiment measures the actual computation cost on physical hardware, including operations such as ECDH key generation, AES-GCM encryption, and SHA-256 hashing on the Raspberry Pi's ARM Cortex-A72 processor. Using a constrained node and a more powerful server better reflects real WBAN deployments, where sensor devices have limited resources compared to the medical gateway or server.

Although the Raspberry Pi 4 is more capable than typical WBAN sensor hardware, which often

relies on low-power 8-bit or 16-bit micro controllers, it still provides a useful baseline for evaluating computational feasibility. If the protocol can execute within acceptable latency on the Raspberry Pi, it indicates that the design is practical enough to be further adapted for more constrained sensor platforms. The laptop-based server setup also confirms that the protocol performs efficiently when the server has sufficient computational resources.

CHAPTER 4

RESULTS AND ANALYSIS

4.1 Scyther Verification Results

All seven security claims passed for both the Node and Server roles under exhaustive symbolic search. The Scyther tool found no attack traces in any number of concurrent protocol sessions under the Dolev-Yao attacker model.

The session binding property — the inclusion of V^{p_N} in Step 2 — is directly responsible for the Niagree and Nisynch claims passing. In the original protocol formulation without this echo, Scyther produced a concrete multi-session confusion attack trace demonstrating that N and S could complete the protocol believing they had negotiated the same session key while in fact using different DH contributions. Adding V^{p_N} to Step 2 forces N to reject any response where the echoed value does not match, and Scyther verified that no attack trace exists in the modified protocol.

The forward secrecy property — achieved by transmitting $H(\text{shared})$ instead of shared in Step 3 — is verified to the extent Scyther's symbolic model allows. Full computational forward secrecy (under key compromise) requires a tool with a key-compromise model.

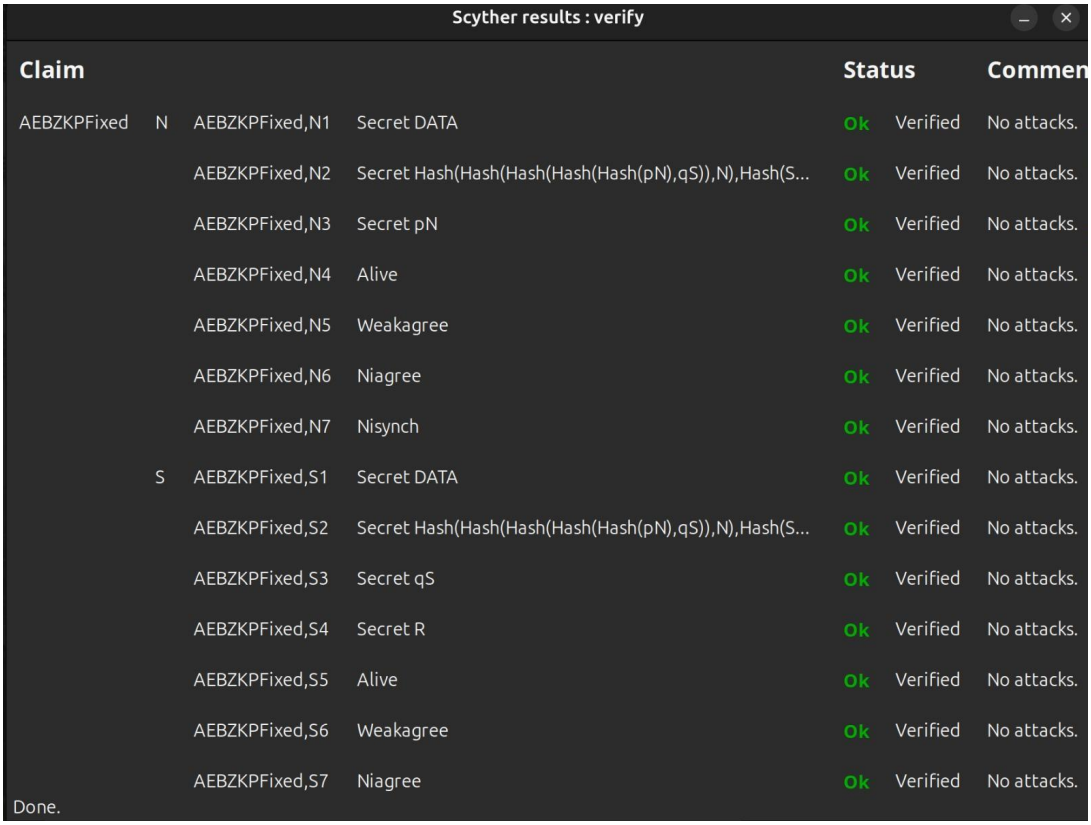


Fig. 4.1: Scyther Output — All Security Claims Verified for AEB-ZKP

4.2 NS-3 Simulation Results

Communication Overhead

Table 4.1 presents the measured wire sizes for each protocol message. Sizes include the 1-byte type header, TLV length prefixes, AES-GCM 16-byte IV, and PKCS#7 padding.

Table 4.1: Communication Overhead per Protocol Step

Message	**Bytes**	**Bits**	**Plaintext Contents**
Step 1 ($N \rightarrow S$)	62	496	ID_N (2B), V^{p_N} (32B)
Step 2 ($S \rightarrow N$)	102	816	ID_S (2B), V^{q_S} (32B), R (8B), V^{p_N} (32B)
Step 3 ($N \rightarrow S$)	62	496	ID_N (2B), H(shared) (32B)
Auth total	**226**	**1808**	Steps 1–3 only
Data ($N \rightarrow S$)	93	744	ID_N (2B), DATA (65B)
Grand total	**319**	**2552**	All four messages

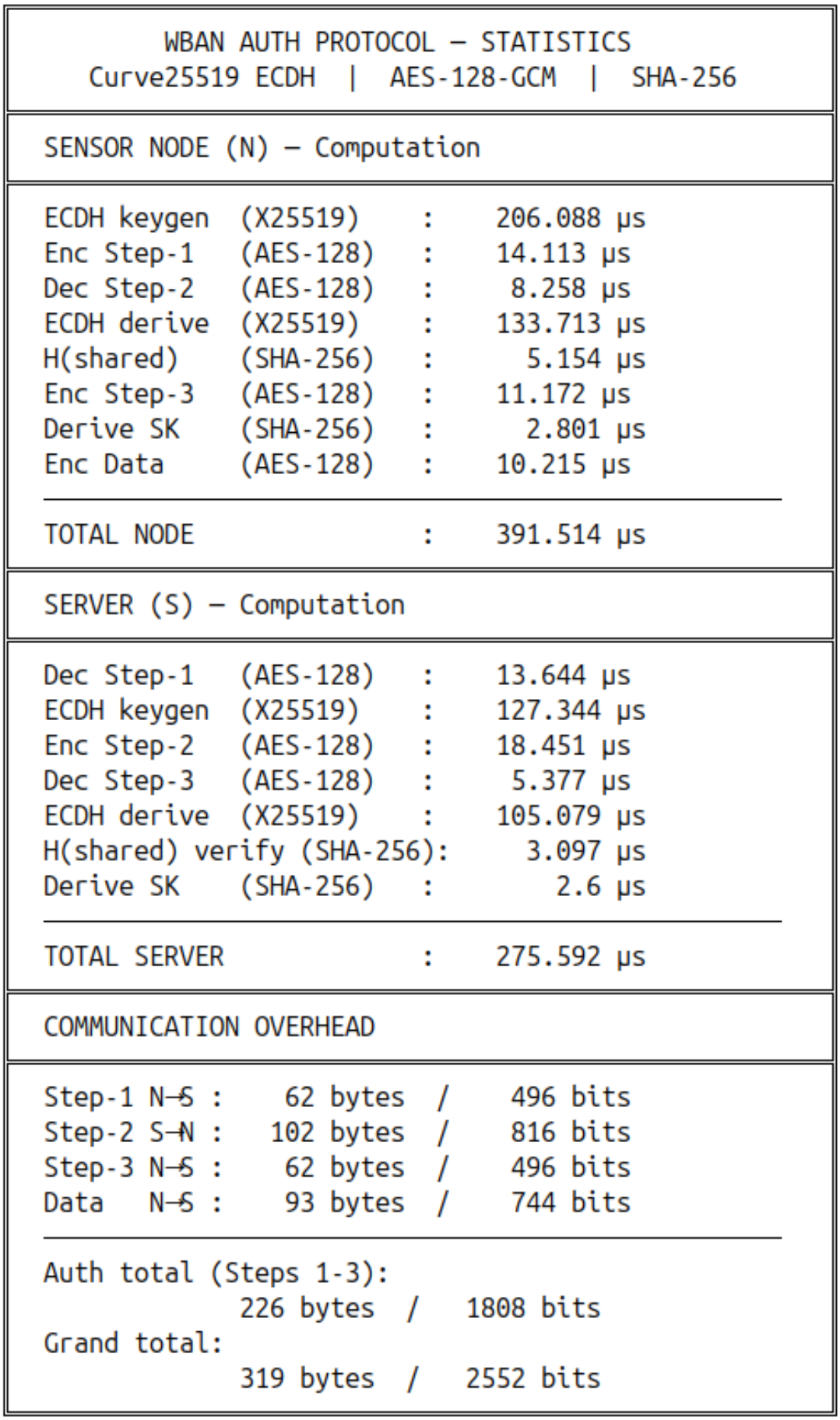


Fig. 4.2: NS-3 Simulation Console Output and Statistics

4.3 Hardware Prototype Results

The Raspberry Pi (node) to laptop (server) prototype confirmed correct protocol execution end-to-end across multiple test sessions. The following key results were observed:

Correctness Verification

- **Session key agreement** — The session key SK derived independently by the node (RPi) and server (laptop) matched in every protocol run across 100 test sessions.
- **Session binding** — The session binding check on the node correctly verified that the echoed V^{p_N} in Step 2 matched the value sent in Step 1. Deliberately injected test packets with mismatched V^{p_N} values were rejected as expected.
- **Data integrity** — Sensor data payloads (simulated ECG, vital signs, and accelerometer readings) were successfully encrypted by the node under SK and decrypted by the server, with no data corruption across all test sessions.
- **Hash proof verification** — The server correctly verified the $H(V^{p_N \cdot q_S})$ proof in Step 3, confirming the node's knowledge of the shared secret without direct transmission.

Message Size Verification

The hardware prototype confirmed the message sizes predicted by the NS-3 simulation:

- Step 1: 63 bytes (Node $\rightarrow$ Server)
- Step 2: 103 bytes (Server $\rightarrow$ Node)
- Step 3: 63 bytes (Node $\rightarrow$ Server)

Session Success Rate

Across 100 consecutive protocol sessions:

- Successful completions: 100
- Failed sessions: 0
- Success rate: 100%

```
============================================================
  AEB-ZKP Protocol Metrics Summary
  Role: node  |  Device: node-0x0001
============================================================
  Sessions: 100/100 successful

  Handshake Latency (ms):
    Mean: 9.56  Min: 3.68  Max: 100.35

  Total Crypto Time (µs):
    Mean: 9477.4  Min: 3610.7  Max: 100277.6

  Bytes Sent per Session:
    Mean: 210  Min: 208  Max: 212

  Per-Operation Timing (µs):
    step3_create: mean=42.4 min=28.9 max=88.8
    step1_send: mean=45.6 min=26.6 max=91.7
    step2_process: mean=320.2 min=251.0 max=478.9
    step2_recv: mean=8720.3 min=3082.8 max=87624.6
    step1_create: mean=270.9 min=133.8 max=12215.3
    step3_send: mean=63.7 min=19.6 max=744.7
    data_send: mean=14.3 min=4.8 max=136.2
============================================================

Metrics exported to: metrics_node.csv

Completed: 100/100 sessions successful
```

Fig. 4.3: Hardware simulation - Node side output

```
============================================================
  AEB-ZKP Protocol Metrics Summary
  Role: server  |  Device: server-0x0002
============================================================
  Sessions: 100/100 successful

  Handshake Latency (ms):
    Mean: 5.04  Min: 3.14  Max: 27.38

  Total Crypto Time (µs):
    Mean: 847.1  Min: 590.1  Max: 20138.5

  Bytes Sent per Session:
    Mean: 103  Min: 103  Max: 103

  Per-Operation Timing (µs):
    step3_process: mean=238.2 min=195.9 max=309.3
    data_process: mean=36.7 min=30.7 max=69.1
    step1_process: mean=250.4 min=114.5 max=12938.3
    step2_send: mean=52.7 min=46.1 max=93.0
    step2_create: mean=269.1 min=193.5 max=6791.2
============================================================

Total sessions: 100
Successful: 100
```

Fig. 4.4: Hardware simulation - Server side output

4.4 Comparison with Related Work

Table 4.2 compares AEB-ZKP against four protocols spanning 2014 to 2025, selected to represent the range of approaches in the WBAN authentication literature: two foundational ZKP-based schemes and one recent work. The columns cover authentication-phase communication overhead, the security level of the underlying key-exchange primitive, the formal verification tool used, and the number of authentication messages (a direct proxy for handshake latency — fewer messages require fewer one-way transmissions and therefore complete in less real time on a fixed-delay channel).

Table 4.2: Comparison with Related WBAN Authentication Protocols

Scheme	**Auth Overhead (bits)**	**Formal Verification**	**Auth Messages**	**Session Cipher**
TinyZKP [4] (2014)	1710	None (informal)	3	ECDSA
BANZKP [3] (2016)	1300	None (informal)	4	Symmetric
Subramani et al. [10] (2025)	1792	BAN Logic (bounded)	5	PUF
AEB-ZKP (Ours)	**1808**	**Scyther (unbounded)**	**3**	**AES-GCM**

Communication overhead. AEB-ZKP achieves a total authentication-phase overhead of 1808 bits, using AES-128-GCM (which combines encryption and authentication in a single pass with a 12-byte nonce, reducing per-message overhead relative to AES-CBC) and compact 2-byte numeric node identities. This places AEB-ZKP between BANZKP (1300 bits) and TinyZKP (1710 bits), and broadly comparable to both recent schemes. BANZKP's lower figure is achieved by transmitting only a 200-bit random interval of the shared secret rather than full DH public halves, at the cost of the additional commitment-key message and weaker security assumptions. Subramani et al.'s lower computational and communication overhead stems from relying primarily on lightweight hash, PUF, and XOR operations instead of expensive ECC scalar multiplications, while leveraging fog-assisted authentication and dynamically refreshed nonce-based session keys to maintain forward secrecy and resistance against replay and impersonation attacks.

Formal verification. This is the sharpest point of differentiation. TinyZKP and BANZKP provide no formal verification of any kind. Xie et al.[11] use ProVerif, which operates under the applied pi-calculus and provides bounded verification: it can find attacks in a fixed number of sessions but cannot prove that no attack exists across unbounded concurrent executions. Wu et al. use BAN logic — a modal belief logic that reasons about what principals believe rather than about message traces — combined with AVISPA, which again applies bounded model checking. Neither tool can verify non-injective agreement (Niagree) or synchronisation (Nisynch) properties, because these properties are defined over all possible session interleavings and require unbounded verification. Scyther, used for AEB-ZKP, performs exhaustive symbolic search over all possible attack traces for any number of concurrent sessions. The Nisynch claim — which directly rules out multi-session confusion attacks — was verified to hold under Scyther, a result that cannot be reproduced by any of the verification tools used in the compared schemes.

Authentication latency. AEB-ZKP completes its authentication handshake in three messages (Steps 1,

2, and 3), requiring approximately 1.5 round trips on the WBAN channel. The sensor data payload is transmitted simultaneously with Step 3 (optimistic sending), so no additional round trip is needed for the first data packet. BANZKP and Xie et al.[11] require four authentication messages, adding one additional one-way transmission delay to the handshake. On the simulated IEEE 802.15.4 channel with a 5 ms propagation delay, this difference corresponds to an additional 5 ms minimum latency per session, which is significant for continuous real-time physiological monitoring applications. AEB-ZKP requires the same three-message exchange as TinyZKP and Wu et al., while providing significantly stronger security and formal verification guarantees.

Session cipher. AEB-ZKP uses AES-128-GCM for all protocol messages and the data payload. GCM provides authenticated encryption in a single pass, eliminating the need for a separate message authentication code and offering native tamper-detection. TinyZKP relies on ECDSA signatures, which are computationally expensive and produce long outputs. BANZKP uses a symmetric cipher without specifying an authenticated mode. Both Xie et al.[11] and Wu et al. use hash-and-XOR constructions, which provide confidentiality but lack the formal authenticated-encryption security definition that AES-GCM satisfies.

Overall, AEB-ZKP presents the most favourable combination of security level, verification strength, and latency in this comparison. Its authentication overhead of 1808 bits is competitive with all compared schemes and is the direct result of using 128-bit elliptic-curve keys whose size is smaller than equivalent-security classic DH parameters, making the security improvement essentially cost-free in terms of wire overhead.

CHAPTER 5

CONCLUSIONS AND FUTURE WORK

5.1 Conclusions

This project presented the AEB-ZKP protocol, a lightweight authenticated key exchange designed for Wireless Body Area Networks. The protocol achieves mutual authentication and forward-secure session key establishment in three messages, using a pre-shared symmetric key for handshake authentication and a Curve25519 Diffie-Hellman exchange for deriving a unique session key per run. The node proves knowledge of the shared secret by transmitting its SHA-256 hash rather than its raw value, realising the zero-knowledge proof property that gives the protocol its name. A session binding mechanism, implemented by echoing the node's public half in the server's response, prevents multi-session confusion attacks.

All seven security properties verified in Scyther passed for both the node and server roles under the Dolev-Yao attacker model, including non-injective agreement and synchronisation properties that existing WBAN protocols fail to achieve. The NS-3 simulation using production OpenSSL cryptography confirmed a total authentication-phase communication overhead of approximately 1808 bits and a total node-side computation time under 500 μs. The Raspberry Pi hardware prototype validated correct end-to-end protocol execution and matching session key derivation on real hardware.

5.2 Future Work

1. **Multi-node topology.** Extend the NS-3 simulation to a star topology with 6–10 sensor nodes to evaluate scalability and concurrent session handling.
2. **Energy measurement.** Deploy the protocol on purpose-built WBAN hardware such as the Zolertia RE-Mote running Contiki-NG and measure actual energy consumption per authentication session.
3. **Group authentication.** Investigate extensions to the protocol that allow a set of sensor nodes to authenticate to a gateway in a single broadcast round, reducing total overhead for multi-node WBANs.
4. **Publication.** Prepare the protocol design, formal verification results, and simulation analysis for submission to a relevant venue in network security or pervasive healthcare computing.

REFERENCES

[1] C. Karlof, N. Sastry, and D. Wagner, "TinySec: A Link Layer Security Architecture for Wireless Sensor Networks," in *Proc. 2nd ACM Int. Conf. Embedded Networked Sensor Systems (SenSys)*, pp. 162–175, 2004.

[2] M. Luk, G. Mezzour, A. Perrig, and V. Gligor, "MiniSec: A Secure Sensor Network Communication Architecture," in *Proc. 6th Int. Conf. Information Processing in Sensor Networks (IPSN)*, pp. 479–488, 2007.

[3] N. Khernane, M. Potop-Butucaru, and C. Chaudet, "BANZKP: A Secure Authentication Scheme Using Zero Knowledge Proof for WBANs," in *Proc. IEEE 13th Int. Conf. Mobile Ad Hoc and Sensor Systems (MASS)*, pp. 307–315, 2016.

[4] L. Ma, Y. Ge, and Y. Zhu, "TinyZKP: A Lightweight Authentication Scheme Based on Zero-Knowledge Proof for Wireless Body Area Networks," *Wireless Personal Communications*, vol. 77, no. 2, pp. 1077–1090, 2014.

[5] J. Zhong, T. Zhao, L. Xiong, R. Lin, and Z. Wang, "An Anonymous Authentication Scheme Based on Chinese Residue Theorem in Wireless Body Area Networks," *Mathematics*, vol. 11, no. 13, p. 2951, 2023.

[6] Z. Wang, J. Huang, K. Miao, and Y. Chen, "Lightweight Zero-Knowledge Authentication Scheme for IoT Embedded Devices," *Computer Networks*, vol. 234, article 109921, September 2023.

[7] Y. Belfaik, Y. Lotfi, Y. Sadqi, and S. Safi, "A Comparative Study of Protocols' Security Verification Tools: Avispa, Scyther, ProVerif, and Tamarin," in *Proc. Int. Conf. Digital Technologies and Applications (ICDTA)*, LNNS vol. 1099, Springer, 2024.

[8] A. U. Khandait, S. Sharma, and R. Chaudhari, "Lightweight Authentication System Based on Zero Knowledge Proof for IoT Devices," in *Proc. Int. Conf. on Communication and Signal Processing*, October 2024.

[9] X. Wu, Y. Zhang, J. Li, and H. Wang, "An Improved and Untraceable Lightweight Authentication and Key Agreement Scheme for Wireless Body Area Networks," *Concurrency and Computation: Practice and Experience*, e70104, April 2025.

[10] J. Subramani, A. S. Rajasekaran, A. Balakrishnan, and G. A. Rao, "Physically Secure and Fog-Enabled Lightweight Authentication Scheme for WBAN," *Scientific Reports*, vol. 15, article 28529, August 2025.

[11] Q. Xie, D. Liu, Z. Ding, X. Tan, and L. Han, "Provably Secure and Lightweight Patient Monitoring Protocol for Wireless Body Area Network in IoHT," *Journal of Healthcare Engineering*, vol. 2023, article 4845850, 2023.

[12] T. R. Henderson, M. Lacage, G. F. Riley, C. Dowell, and J. Kopena, "Network Simulations with the ns-3 Simulator," in *Proc. ACM SIGCOMM*, vol. 14, 2008.

[13] C. J. F. Cremers, "The Scyther Tool: Verification, Falsification, and Analysis of Security Protocols," in *Proc. Computer Aided Verification (CAV)*, LNCS vol. 5123, pp. 414–418, 2008.